Signaturen relevanter Bibliotheksfunktionen

int **pthread_create**(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine) (void*), void *arg);
int **pthread_attr_init**(pthread_attr_t *attr);
int **pthread_attr_destroy**(pthread_attr_t *attr);
void **pthread_exit**(void *value_ptr);
int **pthread_join**(pthread_t thread, void **value_ptr);
int **pthread_detach**(pthread_t thread);
int **pthread_cancel**(pthread_t thread);
int **pthread_setcancelstate**(int state, int *oldstate);
int **pthread_setcanceltype**(int type, int *oldtype);
void **pthread_testcancel**(void);
int **pthread_mutex_init**(pthread_mutex_t *mutex, pthread_mutexattr_t *attr);
int **pthread_mutex_destroy**(pthread_mutex_t *mutex);
int **pthread_mutexattr_init**(pthread_mutexattr_t *attr);
int **pthread_mutexattr_destroy**(pthread_mutexattr_t *attr);
int **pthread_mutexattr_settype**(pthread_mutexattr_t *attr, int type);
int **pthread_mutexattr_gettype**(const pthread_mutexattr_t *attr, int *type);
int **pthread_mutex_lock**(pthread_mutex_t *mutex);
int **pthread_mutex_trylock**(pthread_mutex_t *mutex);
int **pthread_mutex_unlock**(pthread_mutex_t *mutex);
int **pthread_cond_init**(pthread_cond_t *cond, const pthread_condattr_t *attr);
int **pthread_cond_destroy**(pthread_cond_t *cond);
int **pthread_condattr_init**(pthread_condattr_t *attr);
int **pthread_condattr_destroy**(pthread_condattr_t *attr);
int **pthread_cond_wait**(pthread_cond_t *cond, pthread_mutex_t *mutex);
int **pthread_cond_timedwait**(pthread_cond_t *cond, pthread_mutex_t *mutex, const struct timespec *abstime);
int **pthread_cond_signal**(pthread_cond_t *cond);
int **pthread_cond_broadcast**(pthread_cond_t *cond);
int **sem_init**(sem_t *sem, int pshared, unsigned int value);
int **sem_destroy**(sem_t *sem);
int **sem_wait**(sem_t *sem);
int **sem_trywait**(sem_t *sem);
int **sem_post**(sem_t *sem);
sem_t ***sem_open**(const char *name, int oflag);
sem_t ***sem_open**(const char *name, int oflag, mode_t mode, unsigned int value);
int **sem_close**(sem_t *sem);
int **sem_unlink**(const char *name);
int **pthread_barrier_init**(pthread_barrier_t *restrict barrier, const pthread_barrierattr_t *restrict attr, unsigned count);
int **pthread_barrier_destroy**(pthread_barrier_t *barrier);
int **pthread_barrier_wait**(pthread_barrier_t *);
int **pthread_rwlock_init**(pthread_rwlock_t *rwlock, const pthread_rwlockattr_t *attr);
int **pthread_rwlock_destroy**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_rdlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_wrlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_tryrdlock**(pthread_rwlock_t *rwlock);
int **pthread_rwlock_trywrlock**(pthread_rwlock_t *rwlock);

int **pthread_rwlock_unlock**(pthread_rwlock_t *rwlock);
pid_t **getpid**(void);
pid_t **getppid**(void);
int **system**(const char *command);
pid_t **fork**(void);
int **execl**(const char *path, const char *arg0, ... /*, (char *)0 */);
int **execv**(const char *path, char *const argv[]);
int **execle**(const char *path, const char *arg0, ... /*, (char *)0, char *const envp[]*/);
int **execve**(const char *path, char *const argv[], char *const envp[]);
int **execlp**(const char *file, const char *arg0, ... /*, (char *)0 */);
int **execvp**(const char *file, char *const argv[]);
void **exit**(int status);
void **abort**(void);
void ***memset**(void *str, int c, size_t n);
void ***memcpy**(void *dest_str, const void * src_str, size_t n);
pid_t **wait**(int *stat_loc);
pid_t **waitpid**(pid_t pid, int *stat_loc, int options);
int **pipe**(int fd[2]);
int **dup**(int fd);
int **dup2**(int oldfd, int newfd);
FILE ***popen**(const char *cmdstring, const char *type);
int **pclose**(FILE *fp);
int **mkfifo**(const char *pathname, mode_t mode);
mqd_t **mq_open**(const char *name, int oflag);
mqd_t **mq_open**(const char *name, int oflag, mode_t mode, struct mq_attr *attr);
mqd_t **mq_getattr**(mqd_t mqdes, struct mq_attr *attr);
mqd_t **mq_setattr**(mqd_t mqdes, struct mq_attr *newattr, struct mq_attr *oldattr);
int **mq_send**(mqd_t mqdes, const char *msg_ptr, size_t msg_len, unsigned int msg_prio);
ssize_t **mq_receive**(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned int *msg_prio);
int **mq_close**(mqd_t mqdes);
int **mq_unlink**(const char *name);
int **mq_notify**(mqd_t mqdes, const struct sigevent *notification);
int **kill**(pid_t pid, int signo);
int **raise**(int signo);
int **sigaction**(int signo, const struct sigaction *act, struct sigaction *oact);
int **sigprocmask**(int how, const sigset_t *set, sigset_t *oset);
int **pthread_sigmask**(int how, const sigset_t *set, sigset_t *oset);
int **sigpending**(sigset_t *set);
int **sigemptyset**(sigset_t *set);
int **sigfillset**(sigset_t *set);
int **sigaddset**(sigset_t *set, int signo);
int **sigdelset**(sigset_t *set, int signo);
int **sigismember**(const sigset_t *set, int signo);
int **pause**(void);
int **sigsuspend**(const sigset_t *sigmask);
int **sigwait**(const sigset_t *set, int *sig);
int **open**(const char *path, int oflags);
int **open**(const char *path, int oflags, mode_t mode);
ssize_t **read**(int filedes, void *buf, size_t nbytes);
ssize_t **write**(int fildes, const void *buf, size_t nbyte);

int **close**(int filedes);
int **fcntl**(int fildes, int cmd, ...);
off_t **lseek**(int filedes, off_t offset, int whence);
int **ioctl**(int fildes, int request, ... /* arg */);
int **socket**(int *domain*, int *type*, int *protocol*);
int **bind**(int *sockfd*, const struct sockaddr **my_addr*, socklen_t *addrlen*);
uint16_t **htons**(uint16_t hvalue);
uint32_t **htonl**(uint32_t hvalue);
uint16_t **ntohs**(uint16_t hvalue);
uint32_t **ntohl**(uint32_t hvalue);
int **sendto**(int sockfd, const void *buff, size_t nbytes, int flags, const struct sockaddr *to, socklen_t tolen);
int **select**(int maxfdp1, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
int **recvfrom**(int sockfd, void *buff, size_t nbytes, int flags, struct sockaddr *from, socklen_t *fromlen);
in_addr_t **inet_addr**(const char *dotted);
char ***inet_ntoa**(struct in_addr network);
int **listen**(int sockfd, int backlog);
int **connect**(int sockfd, const struct sockaddr *serv_addr,socklen_t addrlen);
int **accept**(int sockfd, struct sockaddr *addr,socklen_t *addrlen);
int **close**(int sockfd);
int **shutdown**(int sockfd, int howto);
FD_ZERO (fd_set *set);
FD_SET (int fd, fd_set *set);
FD_CLR (int fd, fd_set *set);
FD_ISSET(int fd, fd_set *set);
int **getsockname**(int socket, struct sockaddr *restrict address, socklen_t *restrict address_len);

X64 Assembler Ergänzung

DIV x64	Divide (unsigned)	`DIV Op`	`Op=quad.: RAX:=RDX:RAX / Op RDX := Rest`
IDIV x64	Signed Integer Divide	`IDIV Op`	`Op=quad.: RAX:=RDX:RAX / Op RDX := Rest`
MUL x64	Multiply (unsigned)	`MUL Op`	`Op=quad: RDX:RAX:=RAX*Op`
IMUL x64	Signed Integer Multiply	`IMUL Op`	`Op=quad: RDX:RAX:=RAX*Op`